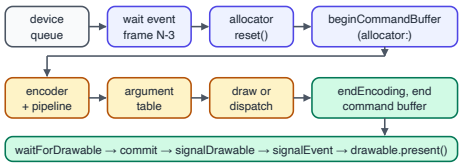
Metal Cheat Sheet

Metal 4 · MSL 4.1 · Xcode 27 · checked 1 November 2026 · Irurueta Docs, Graphics & Games

Version matrix and 2026 baseline

Item	Status
Metal 4	26 OSes (iOS, iPadOS, macOS, tvOS, visionOS); `MTL4*` types adopted next to `MTL*`
MSL	**4.1** (`version4_1`, 27 OSes); 4.0 from 26
Hardware	A14 or later, Apple-silicon Mac (M1+), Apple Vision Pro; hardware ray tracing A17 / M3+
Families	`apple7` A14/M1, `apple8` A15/A16/M2, `apple9` A17/M3/M4; `.metal3`, `.metal4`
Toolchain	**Xcode 27**, Swift 6.4, Apple-silicon Mac on macOS 26.6+; Metal Toolchain is a separate component
Intel	`managed`, `mac2`, `common3` deprecated in 27 OSes

Metal 4 frame loop



Three frames in flight: one allocator and one set of per-frame buffers each. Residency sets are added to the queue once.

MTL to MTL4 type map

Metal 3	Metal 4
`MTLCommandQueue`	`MTL4CommandQueue` (`makeMTL4CommandQueue()`)
`MTLCommandBuffer`	`MTL4CommandBuffer`, made once, reused
(managed by Metal)	`MTL4CommandAllocator`, one per frame
render encoder	`MTL4RenderCommandEncoder`
compute, blit, accel. struct encoders	`MTL4ComputeCommandEncoder`
`setBuffer`, `setTexture`	`MTL4ArgumentTable`
device library calls	`MTL4Compiler`
(none)	`MTL4MachineLearningCommandEncoder`, `MTLTensor`

MSL quick syntax

```
#include <metal_stdlib>
using namespace metal;
struct VOut { float4 pos [[position]]; float2 uv; };
vertex VOut vs(uint vid [[vertex_id]],
  constant Uniforms &u [[buffer(0)]],
  device const Vert *v [[buffer(1)]]) {...}
fragment float4 fs(VOut in [[stage_in]],
  texture2d<float> t [[texture(0)]],
  sampler s [[sampler(0)]]) {
  return t.sample(s, in.uv); }
kernel void k(uint2 gid [[thread_position_in_grid]]);
constant bool useFog [[function_constant(0)]];
[[stitchable]] float4 effect(float4 c);
```

Address spaces: `device` read/write GPU memory, `constant` read-only, `threadgroup` shared in a group, `thread` private. Tile: `[[color(n)]]`, `imageblock`.

Storage modes

Mode	Use it for
`.storageModeShared`	CPU writes, GPU reads; the Apple-silicon default for buffers and per-frame data
`.storageModePrivate`	GPU only: static meshes, render targets, copied in with a blit
`.memoryless`	Tile memory only: depth, G-buffer, MSAA attachments that never need saving
`.managed`	Intel and discrete-GPU Macs; deprecated in 27 OSes

Load and store actions, TBDR tips

Load	Store
`.clear`: start from a value	`.store`: write tile to memory
`.dontCare`: overwritten anyway	`.dontCare`: never read later
`.load`: read earlier contents	`.multisampleResolve`, `.storeAndMultisampleResolve`

- Clear instead of load; `.dontCare` for depth after the pass.
- Keep G-buffers `.memoryless` with tile shaders and imageblocks.
- Do many draws per pass: every pass boundary flushes tiles to memory.

Compute dispatch

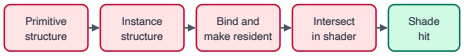
```
enc.setComputePipelineState(pso)
let w = pso.threadExecutionWidth        // SIMD width
let h = pso.maxTotalThreadsPerThreadgroup / w
enc.dispatchThreads(
  MTLSize(width: tex.width, height: tex.height, depth:
1),
  threadsPerThreadgroup: MTLSize(width: w, height: h,
depth: 1))
```

Grid → threadgroups → SIMD groups → threads. Up to 1,024 threads per group on A11 and later (check the Feature Set Tables). `threadgroup` memory plus `threadgroup_barrier` for sharing.

Resource binding

- **Argument table** (`MTL4ArgumentTable`): `setAddress(_:index:)`, `setResource(_:bufferIndex:)`, `setSamplerState(_:index:)`; `setArgumentTable(_:stages:)` on the encoder.
- **Bindless**: write `gpuResourceID` (textures, samplers) and `gpuAddress` (buffers) into a buffer; read them as a struct in MSL.
- **Residency**: `MTLResidencySet.addAllocation`, `commit()`, `queue.addResidencySet`; the layer has its own.
- **Heaps** (`MTLHeap`) suballocate buffers and textures; add the heap, not each resource.
- Argument encoders (`MTLArgumentEncoder`) still work.

Ray tracing steps



1. Describe geometry (`MTLAccelerationStructureTriangleGeometryDescriptor`) and build the primitive structure; compact it.
2. Build the instance structure (transforms, masks, user IDs).
3. Pass it in the argument table; add it to a residency set.
4. `intersector<triangle_data, instancing>` for the closest hit, or `intersection_query` to walk candidates.

Hardware on A17 / M3 and later. `MPSRayIntersector` is deprecated.

Mesh-shader pipeline

`[[object]]` function (one threadgroup per task: culls, picks the LOD, sets `mesh_grid_properties`) → `[[mesh]]` function (writes a meshlet of vertices and primitives) → rasteriser → fragment. Encode with `drawMeshThreadgroups`. iOS 16 / macOS 13. Replaces tessellation for amplification and LOD; Metal 4 render encoders have no tessellation setup.

MetalFX options

Effect	Needs
`MTLFXSpatialScaler`	colour only
`MTLFXTemporalScaler`	colour, depth, motion vectors, jitter, exposure
`MTLFXTemporalDenoisedScaler`	temporal inputs + normals, albedo, roughness
`MTLFXFrameInterpolator`	temporal scaler, depth, motion, camera matrices, delta time
Neural upscaling, denoising	WWDC26 additions

Jitter the projection with a Halton sequence; render at lower resolution, output at full.

TBDR at a glance



Unified memory and tile memory on Apple GPUs: tile shaders and imageblocks read the G-buffer without leaving the chip.

MetalKit and presentation

- `MTKView` in SwiftUI through a representable; delegate `draw(in:)` and `drawableSizeWillChange`.
- `CAMetalLayer` owns drawables; `CAMetalDisplayLink` (iOS 17, macOS 14) paces the frame.
- `MTKTextureLoader` for images, `MTKMesh` from Model I/O (USD, USDZ).
- Matrices are column-major `simd` types; NDC depth runs 0 to 1.
- visionOS: Compositor Services, `LayerRenderer`, stereo and foveation.

Debugging and profiling tools

- Xcode GPU frame capture: dependency viewer, shader debugger, shader cost graph, heat maps, counters.
- API and shader validation; Metal Performance HUD; Metal System Trace in Instruments.
- CLI (WWDC26): `gpucapture`, `gpudebug`, `metalperftrace`.
- Label objects (`label`, debug groups) so captures read well.

Porting

- **metal-cpp**: header-only C++ wrapper; `NS::`, `MTL::`; mind autorelease pools.
- **Metal shader converter**: DXIL → metallib; Windows tools available.
- **Game Porting Toolkit 4**: evaluate and port DirectX games.
- OpenGL: Apple's migration sample and the concept map.

Deprecations strip

Old	Use
`MTLFeatureSet`	`supportsFamily(_:)`
`MPSRayIntersector`	acceleration structures
PVRTC	ASTC, or BC on Mac
OpenGL / ES	Metal (since iOS 12, macOS 10.14)
`.managed`, `.mac2`, `.common3`	shared, `.apple*` families (27 OSes)
`newDefaultLibrary()`	`makeDefaultLibrary()`